brainimg: A Reproducible Systems Study of Generative-Recall Image Compression

2026-06-21

Abstract

Classical image formats store pixels—either directly, as transform coefficients, or as latent codes—and are therefore resolution-bound and tied to the exact appearance of the original. This paper presents *brainimg*, a small prototype image format that stores the *meaning* of an image (a text caption) plus a tiny *structural blueprint* (128×128 depth, Canny-edge, and ADE20K segmentation maps) and a seed, and regenerates a visually faithful image on decode using one of ten pluggable diffusion decoder backends: Stable Diffusion 1.5 (default) or SDXL with two-to-three ControlNets, either of those plus ByteDance's Hyper-SD 8-step distilled LoRA (turbo), Z-Image-Turbo with a single Union ControlNet, Qwen-Image (Apache 2.0) with InstantX's Union ControlNet, or FLUX.1-Depth-dev / FLUX.1-Canny-dev with channel-concat conditioning, optionally with Hyper-SD's 8-step FLUX LoRA. We frame brainimg as a working, reproducible instantiation of the "Semantic-Relational Field / generative-recall" paradigm: rather than compressing appearance, it stores the scene's semantics and geometry and lets a diffusion model repaint it. We describe the format schema, the four-stage encoder (VLM captioning, depth estimation, edge extraction, semantic segmentation), the ten decoder backends, the per-device memory/precision strategies, and the brightness/saturation post-processing with a gamma fallback for extreme targets. Using measurements reproducible from the committed repository on an AMD x86_64 CPU-only target with 188 GB RAM, we report blueprint sizes of 2.7–7.9 KB (compression ratios of 2.2×–99.7×), deterministic reconstruction given a fixed seed, and a per-backend fidelity and speed comparison on the Lenna test image in which Hyper-SD 8-step distilled schedules *beat* their 30-step counterparts on both SD 1.5 (+0.95 dB) and FLUX (+1.41 dB), and a ControlNet scale sweep finds that lower depth scales (0.8 vs 1.5) improve fidelity with the Depth-Anything-V2-Base stack. We are explicit that brainimg is lossy-by-design, decoder-dependent, and unsuited to forensic/medical use; we position it as a systems study of a novel paradigm rather than a replacement for JPEG.

1. Introduction

Every widely used image format answers a variant of the same question: *how do we store these pixels efficiently?* PNG stores exact colors and predicts the next pixel; JPEG/WebP discard transform coefficients the human visual system barely notices; AVIF generalizes this to AV1 intra-frames. All three paradigms share two properties: the file is bound to a native resolution, and "loss" manifests as visible artefacts—blockiness, ringing, colour bleeding.

Neuroscience offers a different model. The retina compresses roughly 130 million photoreceptor signals into about one million optic-nerve fibres (a ~130:1 ratio) by transmitting *edges and contrast* rather than absolute brightness. Deeper in the visual cortex, representation becomes increasingly semantic—neurons respond to faces, objects, materials—until recall itself becomes a generative act: to imagine an apple is not to "open a file" but to drive top-down signals from concept areas back into the visual cortex and *hallucinate* the sensory experience back into existence. The brain stores meaning and the rules to rebuild, not a pixel grid.

This paper is a reproducible systems study of a compression paradigm that mimics that idea. We ask:

> **Can a modern diffusion model serve as the decoder for a kilobyte-scale image format, conditioned on a text caption and a handful of low-resolution structural maps?**

We do **not** claim a new state of the art, nor that the result replaces JPEG. We claim that the paradigm is implementable today, on commodity hardware (including an 8 GB Apple Silicon laptop), that it is deterministic given a seed, and that it exhibits qualitatively different—and in some ways useful—failure modes from transform codecs.

Contributions

1. **Paradigm framing.** We articulate the "Semantic-Relational Field / generative-recall" image-compression paradigm and map it onto a concrete encoder–format–decoder architecture.
2. **A versioned, forward-compatible file format.** `brainimg` v0.1 is a small JSON document carrying a caption, three optional conditioning maps, a seed, and colour statistics. Optional fields allow older files to decode unchanged as new conditioners are added.
3. **A four-stage encoder** combining a vision-language model (Qwen2-VL on MLX, Qwen2.5-VL-7B on CPU/CUDA), Depth-Anything-V2-Base, OpenCV Canny, and OneFormer ADE20K semantic segmentation, with explicit memory release between stages so the pipeline fits in 8 GB.
4. **Ten pluggable decoder backends** spanning four model families (SD 1.5, SDXL, Z-Image, Qwen-Image, FLUX), each with an optional Hyper-SD 8-step distilled turbo variant for SD 1.5, SDXL, and FLUX. All backends consume the same blueprint; new backends require no schema change.
5. **Empirical findings on distilled schedules and scale tuning.** On the Lenna test image, Hyper-SD's 8-step distilled LoRA *beats* the 30-step non-turbo path on both SD 1.5 (+0.95 dB) and FLUX (+1.41 dB) while running 3.5–4× faster on CPU. A grid sweep of ControlNet conditioning scales across two test images finds that Depth-Anything-V2-Base's sharper depth map over-constrains at the historical default of 1.5; lowering depth to 0.8 and raising seg to parity (1.0) yields +0.51 dB on the turbo path.
6. **A gamma fallback for the brightness-clamp edge case.** When the uniform gain clamp $[0.5, 2.0]$ cannot reach an extreme target, a per-channel gamma curve (same exponent, clamped to $[0.3, 3.0]$) closes the residual gap without the clipping artefacts a >2× gain would cause.

The remainder of the paper is organised as follows. §2 surveys related work in classical and learned compression, text-to-image generation, and ControlNet conditioning. §3 describes the method (format, encoder, decoder, device strategy). §4 reports measurements reproducible from the committed repository. §5 discusses the paradigm's properties—resolution independence, semantic loss, decoder dependency—and honest limitations. §6 concludes and points to planned work.

2. Related Work

2.1 Classical image codecs

The three dominant compression paradigms are *spatial/predictive* (PNG, run-length + DEFLATE over filtered scanlines), *spectral/transform* (JPEG's discrete cosine transform, WebP's VP8 intra, AVIF's AV1 intra), and *fractal/wavelet* (JPEG 2000). All are pixel-bound: the file encodes a fixed grid at a fixed resolution, and "loss" is a transform artefact. brainimg departs from all of these by storing *no pixel data at all*.

2.2 Learned / neural compression

Neural compression typically maps pixels to a lower-dimensional latent space via an autoencoder and entropy-codes the latents. The decoder is a learned up-sampler. brainimg is adjacent but distinct: its "latent" is not a continuous vector but a *human-readable caption plus a few low-resolution maps and a seed*, and the decoder is a general-purpose text-to-image diffusion model conditioned by ControlNets rather than a codec-specific autoencoder. The decoder is therefore large (~4 GB for SD 1.5) and shared across all files, while each file is tiny and self-describing.

2.3 Text-to-image diffusion and ControlNet

Stable Diffusion 1.5 [Rombach et al., 2022] performs latent diffusion conditioned on a CLIP text embedding. ControlNet [Zhang et al., 2023] adds a trainable copy of the UNet encoder blocks that forces the diffusion process to follow an external spatial conditioner (depth, edges, segmentation, pose). brainimg uses two ControlNets unconditionally (depth + Canny) and a third (ADE20K segmentation) when the blueprint carries a seg map. The combination of CLIP caption conditioning with multi-map ControlNet conditioning and a stored seed is, to our knowledge, novel as a *file-format* design, though the individual components are off-the-shelf.

2.4 Vision-language captioning

Accurate captions are the semantic backbone of the format. We use the Qwen2-VL-2B-Instruct 4-bit model via MLX on Apple Silicon for fast, low-memory captioning, and fall back to the Qwen2.5-VL-7B-Instruct model via transformers elsewhere. A known failure mode (§4.5) is that the captioner can misidentify scene elements; because the conditioning maps drive structural fidelity, a wrong caption biases mood more than geometry.

2.5 The biological inspiration

The retina's ~130:1 compression via lateral inhibition, sparse coding in V1, and top-down generative recall are well documented in the neuroscience literature. We use these only as *motivation*; brainimg is an engineering artefact, not a brain model.

3. Method

brainimg has three parts: an *encoder* that extracts a blueprint from a source image, a *file* that stores it, and a *decoder* that regenerates an image from the blueprint. Encoder and decoder run as **separate processes** so their heavy models are never resident simultaneously—an important constraint on the 8 GB Apple Silicon target.

```
Source image --> ENCODER --> .brainimg (JSON, ~3-10 KB) --> DECODER --> image
   (jpg/png)      |                                (SD+ControlNet)   (any size)
                    +- 1. Caption (Qwen-VLM)         --> prompt
                    +- 2. Depth   (Depth-Anything-V2) --> 128^2 JPEG
                    +- 3. Canny   (OpenCV)             --> 128^2 PNG
                  +- 4. Seg     (OneFormer ADE20K)  --> 128^2 PNG (optional)
```

3.1 The `.brainimg` file format

A `.brainimg` file is a small UTF-8 JSON document, typically 3–10 KB regardless of source resolution. The schema is versioned (`format_version` is currently `"0.1"`). Required fields are `format_version`, `original_width`, `original_height`, `prompt`, `depth_map_b64`, `canny_map_b64`, and `seed`. Optional fields, added after the initial release, are stored so that older files still decode: `segmentation_map_b64` enables the seg ControlNet only when present and non-empty. The `extra` dictionary absorbs unknown fields for forward compatibility. A representative file:

```
{
  "format_version": "0.1",
  "original_width": 1024, "original_height": 768,
  "prompt": "a red apple on a wooden table next to a window",
  "negative_prompt": "blurry, low quality, deformed, watermark, jpeg artifacts",
  "depth_map_b64": "<base64 128×128 JPEG>",
  "canny_map_b64":  "<base64 128×128 PNG>",
  "segmentation_map_b64": "<base64 128×128 PNG, optional>",
  "seed": 42, "steps": 30,
  "target_brightness": 142.5,
  "target_saturation":  88.3,
  "extra": { "color_style": "vivid, saturated colors, natural lighting, red dominant to
}
```

The format module (`brainimg/format.py`) is deliberately free of any ML import so the schema and round-trip tests run without downloading models. `MAP_SIZE` (currently 128) controls conditioning-map resolution; raising it improves structural fidelity at the cost of a slightly larger file and requires re-encoding existing samples.

3.2 Encoder

The encoder (`brainimg/extract.py`) runs four extractors in sequence, releasing memory between heavy stages via `free_torch()` / `free_mlx()` so the pipeline fits in 8 GB.

Stage 1 — Captioning. A Qwen2-VL vision-language model is prompted with the instruction *"In one short sentence, describe this image for reconstruction: main objects, layout, lighting, and colors."* On Apple Silicon the MLX 4-bit `mlx-community/Qwen2-VL-2B-Instruct-4bit` model is used (fast, low memory); on any other platform the transformers `Qwen/Qwen2.5-VL-7B-Instruct` model is used. Both produce an equivalent caption; the 7B is noticeably more detailed. MLX memory is released before any PyTorch stage runs.

Stage 2 — Depth map. Depth-Anything-V2-Base (`depth-anything/Depth-Anything-V2-Base-hf`) via a HuggingFace `depth-estimation` pipeline produces a grayscale depth map (near = bright, far = dark), downscaled to `MAP_SIZE × MAP_SIZE` with Lanczos resampling and JPEG-encoded at quality 70.

Stage 3 — Canny edges. OpenCV's Canny detector (low=50, high=150) is run on the grayscale image downscaled with `INTER_AREA`, then PNG-encoded to keep edges crisp.

Stage 4 — Segmentation (optional). OneFormer ADE20K (`shi-labs/oneformer_ade20k_swin_ti` produces a semantic map that is colourised with the exact 150-class ADE20K palette the SD 1.5 seg ControlNet was trained on, then nearest-neighbour downscaled to `MAP_SIZE × MAP_SIZE` and PNG-encoded. This stage was added after the initial v0.1 release; older files omit it and decode unchanged.

Colour-style and statistics. Alongside the four conditioners the encoder computes two scalar statistics of the original image, stored for decoder-side post-processing. *Brightness* is the mean Rec.609 luminance on a 0–255 scale:

$$B = \text{mean}(0.299\,R + 0.587\,G + 0.114\,B)\,.$$

Saturation is the mean normalised channel spread, scaled to 0–255 (0 = grey, 255 = fully saturated):

$$S = 255 \cdot \text{mean}\left(\frac{\max_c - \min_c}{\max(\max_c, 1)}\right).$$

A short *colour-style prefix* is also derived (e.g. *"dark, low-key lighting, red dominant tones"*) and stored in `extra`; the decoder prepends it to the caption only when the combined length fits the CLIP 77-token limit, so the caption itself is never truncated.

3.3 Decoder

The decoder (`brainimg/generate.py`) regenerates an image from the blueprint using one of ten pluggable backends, all sharing the same blueprint schema and the same brightness/saturation post-processing:

`--model`	Base	Conditioning	Steps (default)	License
`sd15` (default)	`stable-diffusion-v1-5`	depth + canny (+ seg) ControlNets	30	CreativeML Open RAIL-M
`sd15-turbo`	SD 1.5 + Hyper-SD 8-step LoRA	depth + canny (+ seg) ControlNets	8	CreativeML Open RAIL-M
`sdxl`	`stable-diffusion-xl-base-1.0`	depth + canny (+ seg) ControlNets	30	CreativeML Open RAIL-M
`sdxl-turbo`	SDXL + Hyper-SD 8-step LoRA	depth + canny (+ seg) ControlNets	8	CreativeML Open RAIL-M
`zimage`	`Tongyi-MAI/Z-Image-Turbo`	single Union ControlNet (depth)	9 (8-step Turbo)	Tongyi-MAI non-commercial
`qwen-image`	`Qwen/Qwen-Image`	single Union ControlNet (depth)	50	**Apache 2.0**
`flux-depth`	`black-forest-labs/FLUX.1-Depth-dev`	channel-concat `depth_map_b64`	30	FLUX.1-dev non-commercial
`flux-canny`	`black-forest-labs/FLUX.1-Canny-dev`	channel-concat `canny_map_b64`	30	FLUX.1-dev non-commercial
`flux-depth-turbo`	FLUX.1-Depth-dev + Hyper-SD 8-step FLUX LoRA	channel-concat `depth_map_b64`	8	FLUX.1-dev non-commercial
`flux-canny-turbo`	FLUX.1-Canny-dev + Hyper-SD 8-step FLUX LoRA	channel-concat `canny_map_b64`	8	FLUX.1-dev non-commercial

All ten backends consume the **same blueprint**; the schema is unchanged when a new decoder is added. We describe the SD 1.5 / SDXL path in detail (the historical default), then summarise the turbo, Z-Image, Qwen-Image, and FLUX paths that diverge structurally.

3.3.1 SD 1.5 / SDXL (ControlNet stack)

The default backend. SD 1.5 ships at 512^2, SDXL at 1024^2.

Conditioning. The depth and Canny maps are decoded from base64 and upscaled to the target size (Lanczos for depth, nearest-neighbour for Canny and seg to keep

edges/palette crisp). When a segmentation map is present it is appended as a third conditioner. Default conditioning scales for SD 1.5 are depth 0.8, Canny 1.0, seg 1.0 — tuned via a grid sweep on Lenna and test512 (§4.8). The historical defaults (depth 1.5, canny 1.2, seg 0.9) were set for the older Depth-Anything-Small + no-seg pipeline; Depth-Anything-V2-Base's sharper depth map over-constrains at 1.5, and the ADE20K seg ControlNet adds material cues that warrant parity with Canny.

VAE substitution. The stock SD 1.5 VAE is replaced with the fine-tuned `stabilityai/sd-vae-ft-mse` for cleaner decode, better skin tones and colours, and fewer washed-out highlights, at negligible runtime cost.

Scheduler and steps. The pipeline uses a `UniPCMultistepScheduler` with a default of 30 inference steps (raised from 20). ControlNet scales and classifier-free guidance (default 7.5) are CLI-tunable.

Determinism. A `torch.Generator` seeded with `data.seed` makes generation deterministic: re-decoding with the same seed reproduces the same image exactly (verified to produce 0 pixel difference between runs).

Colour post-processing. SD 1.5 systematically over-brightens and over-saturates. The decoder corrects this against the stored targets in two ordered steps, chosen so each step does not undo the other:

1. *Saturation first*, by scaling the HSV-S channel by `target_saturation / current_saturation`, iterated 1–2× to converge (HSV-S is not linear in the metric above). Ratios are clamped to $[0.5, 2.0]$ to avoid clipping artefacts. This perturbs brightness, which the next step corrects.
2. *Brightness last*, by a uniform RGB gain `target_brightness / current_brightness`. A uniform gain preserves colour balance and leaves the saturation metric (a ratio of channels) invariant, so correcting brightness last does not undo the saturation work. The same $[0.5, 2.0]$ clamp applies.

The step is a no-op for older files whose targets are 0.0, or when the generation's stats are already within ~2 % of the targets. When the clamped gain cannot reach an extreme target (e.g. darkening 210→80 needs ratio 0.38, clamped to 0.5 → 105), a **gamma fallback** applies a per-channel gamma curve arr^{γ} (same exponent on every channel, clamped to $[0.3, 3.0]$) to close the residual gap. Gamma preserves colour balance approximately (not exactly like a uniform gain, but far better than clipping) and reaches extreme targets without the posterisation a >2× gain would cause. The gamma exponent is $\gamma = \log(t/255)/\log(c/255)$ where c and t are the current and target brightnesses normalised to $[0, 1]$.

Style-prefix gating. The stored colour-style prefix is prepended to the caption only when the combined length fits within the CLIP 77-token limit, biasing mood without ever truncating the caption. Older files with no stored style use the caption verbatim. (For Z-Image-Turbo and FLUX, which use a Qwen / T5 text encoder with a 512-token limit, the style prefix is prepended unconditionally.)

3.3.2 Z-Image-Turbo (Union ControlNet)

`--model zimage` swaps the SD stack for **Tongyi-MAI/Z-Image-Turbo** (a 6 B-parameter single-stream DiT distilled for ~8 steps) plus the `alibaba-pai/Z-Image-Turbo-Fun-Controlnet-Union-2.1` *Union* ControlNet (the full `2.1-8steps` variant, ~6.4 GB). The Union net is a single network that takes one conditioning image per call and bakes the conditioning *type* (depth/canny/pose/mlsd/hed) into its training; we feed the blueprint's `depth_map_b64` because depth carries the most structural information. The blueprint's `canny_map_b64` and `segmentation_map_b64` are **silently ignored** on this path—no schema change. Z-Image runs in **bf16 throughout**, which sidesteps the MPS fp16 NaN bug entirely (a different dtype, no int8 quantization is used). Defaults: `guidance_scale = 0.0` (Turbo is distilled for zero CFG), 9 inference steps (8 DiT forward passes on the Turbo schedule), 1024 max side.

3.3.3 FLUX.1-Depth-dev / FLUX.1-Canny-dev (channel-concat)

`--model flux-depth` and `--model flux-canny` load Black Forest Labs' **FLUX.1** guidance-distilled Control variants. FLUX.1 is a 12 B-parameter MMDiT transformer with a T5-XXL text encoder (CLIP-L is the small auxiliary encoder). The FLUX.1-*-dev Control checkpoints bake the conditioning into the transformer via **channel-wise concatenation** of the conditioning image—not a separate ControlNet model. Diffusers' `FluxControlPipeline` takes a single `control_image` per call; we feed `depth_map_b64` for `flux-depth` and `canny_map_b64` for `flux-canny`. The other map and any `segmentation_map_b64` are silently ignored. bf16 throughout (FLUX's native dtype, sidesteps the MPS fp16 NaN bug). Defaults: cfg 10.0 (depth) or 30.0 (canny), 30 inference steps, 1024 max side, `max_sequence_length = 512` (T5-XXL). No per-call `controlnet_conditioning_scale`—the conditioning strength is fixed by the trained checkpoint, so the `--depth-scale` / `--canny-scale` CLI flags are accepted for argument parity but silently ignored on FLUX paths. **License**: FLUX.1-Depth-dev / -Canny-dev carry the FLUX.1-dev non-commercial license.

3.3.4 Hyper-SD turbo distillation (SD 1.5 / SDXL / FLUX)

ByteDance's **Hyper-SD** [Ren et al., 2024] applies trajectory-segmented consistency distillation to produce small (~70–150 MB) LoRAs that fold the SD 1.5, SDXL, and FLUX.1-dev base models down to **8 inference steps** while preserving the stock ControlNets / channel-concat conditioning. The LoRA is loaded via `pipe.load_lora_weights` and fused with `pipe.fuse_lora(0.125)` (per the model card) after device placement, so the UNet / transformer weights are permanently adjusted — no per-call LoRA overhead. For SD 1.5 and SDXL, the scheduler is swapped to `DDIMScheduler(timestep_spacing="trailing")`; for FLUX, no scheduler swap is needed (`FlowMatchEulerDiscreteScheduler` works natively with fewer steps). The turbo paths ignore the file's stored step count (tuned for 20–30 step SD schedules) and use 8 steps unless `--steps` is passed.

FLUX turbo compatibility fix. The Hyper-SD FLUX LoRA was trained on the base `FLUX.1-dev`, not the Control variants. The Control transformer's `x_embedder` (patch embedding) has extra input channels for the control image (128 vs 64) and a `context_embedder` that base dev doesn't have, so those LoRA deltas are

shape-incompatible. The decoder strips the `x_embedder` and `context_embedder` keys (and the `transformer.` prefix, which diffusers adds internally) before loading. The attention/FFN LoRA deltas — the bulk of the distillation — load cleanly and are what matters for the 8-step schedule. Guidance drops to 3.5 (the FLUX dev default, not the 10.0 / 30.0 the non-turbo control variants use).

3.3.5 Qwen-Image (Union ControlNet, Apache 2.0)

`--model qwen-image` loads Alibaba's **Qwen-Image** [Wu et al., 2025] (arXiv 2508.02324, Apache 2.0) — a DiT with a Qwen2.5-VL text encoder (512-token limit) — plus InstantX's `Qwen-Image-ControlNet-Union`, a single Union ControlNet supporting canny + depth + pose + soft-edge. Same pattern as Z-Image: depth-only conditioning, blueprint's canny/seg ignored, bf16 throughout. Defaults: 50 steps, `true_cfg_scale = 4.0` (Qwen-Image's CFG parameter, distinct from `guidance_scale`), `controlnet_conditioning_scale = 0.9`, 1024 max side. The Qwen-Image backend is the only fully-open (Apache 2.0) high-quality option in the stack; FLUX is non-commercial and Z-Image is non-commercial.

3.4 Device and precision strategy

The current dev target is an **AMD x86_64 CPU-only box with 188 GB RAM** (no CUDA/MPS), so the CPU fp32 path is the primary measurement platform. The decoder also supports Apple Silicon (MPS int8) and NVIDIA CUDA (fp16), but those are secondary. On MPS, SD 1.5 fp16 matmuls produce NaNs (a black output frame); the decoder therefore int8-quantizes the UNet and all ControlNets (weights *and* activations) via `optimum-quanto` to avoid fp16 matmuls entirely, while the VAE runs in fp32 for a clean final decode. On CPU, full fp32 is supported (best fidelity, slow) and an optional int8 weights mode fits in ~5 GB at a small quality cost. On CUDA, fp16 works correctly and is fast. Z-Image, Qwen-Image, and FLUX are bf16-native and sidestep the MPS fp16 NaN bug entirely; on a CPU-only box they are kept resident in host RAM (diffusers' `enable_model_cpu_offload` raises `RuntimeError` without an accelerator to offload *to*).

`--device` / `--model`	Precision	RAM	Speed	Fidelity
`cpu` (sd15, default target)	fp32 (no quant)	~10 GB	156 s @ 512²	**best** (sd15)
`cpu --model sd15-turbo`	fp32 + Hyper-SD 8-step LoRA	~10 GB	**50 s** @ 512²	good (+0.95 dB vs 30-step)
`cpu --model sdxl`	fp32 (no quant)	~17 GB	220 s @ 512²	**best** (sdxl)
`cpu --model sdxl-turbo`	fp32 + Hyper-SD 8-step LoRA	~17 GB	**69 s** @ 512²	good (−0.23 dB vs 30-step)
`cpu --model zimage`	bf16 resident	~18 GB	237 s @ 512²	good (depth-only)

`--device` / `--model`	Precision	RAM	Speed	Fidelity
`cpu --model qwen-image`	bf16 resident	~20 GB	1436 s @ 512²	good (depth-only)
`cpu --model flux-depth --quantize`	bf16 + FP8 (host RAM)	~12 GB	654 s @ 512²	**best** (FLUX)
`cpu --model flux-depth-turbo --quantize`	bf16 + FP8 + Hyper-SD 8-step	~12 GB	**166 s** @ 512²	**best+** (+1.41 dB vs 30-step)
`mps` (sd15, Apple Silicon)	int8 weights + activations	~5 GB	medium (8 GB Mac)	fair
`cuda` (sd15)	fp16	~5 GB	**fast**	good

The encoder/decoder device selection is centralised in `brainimg/device.py`, which also provides `free_torch()` / `free_mlx()` memory-release helpers.

4. Experiments

All measurements below are reproducible from the committed repository; we use only data already present in the repo and do not run the heavy ML models here. Where a number is documented in the project (`README.md`, `TODO.md`) we cite it; where it is an on-disk file size we measured it directly with `stat`. We do not report PSNR/SSIM/LPIPS/FID because those runs are out of scope for this writing session; §5 flags the planned evaluation.

4.1 Setup

- **Hardware:** AMD x86_64 CPU-only, 188 GB RAM (the current dev target), 32 cores, 16 torch threads. All measurements in §4 are from this machine. The M1/8 GB Apple Silicon results in `README.md` (59 s @ 256²) are historical and noted where relevant.
- **Software:** Python 3.12, `uv`-managed environment, `requirements.txt`. `diffusers` 0.38, `peft` 0.19 (for LoRA loading), `optimum-quanto` (FP8).
- **Models:** SD 1.5 + `sd-vae-ft-mse` + 3 ControlNets (default), SDXL, Hyper-SD 8-step LoRAs (SD 1.5, SDXL, FLUX), Z-Image-Turbo, Qwen-Image, FLUX.1-Depth-dev / -Canny-dev. Captioner is transformers Qwen2.5-VL-7B (CPU fallback; MLX is Apple-Silicon-only).
- **Samples:** `samples/real.jpg` (256×256 puppy JPEG, 13,430 B), `samples/lenna.tiff` (512×512 Lenna, 786,572 B), `samples/test512.jpg` (512×512, 49,690 B). Blueprints: `real.brainimg`, `lenna.brainimg`, `test512.brainimg`. All Lenna decodes use seed 916570520, 512×512 output.

4.2 Compression

Table 2 reports blueprint sizes and compression ratios against the source files. The "5.0×" figure for the puppy in `README.md` corresponds to an earlier pre-segmentation v0.1 file of ~2.7 KB; the committed `real.brainimg` includes the seg map and colour statistics and is 6,120 B (2.2×), reflecting the richer current schema.

Source file	Source size	`.brainimg` size	Ratio
`samples/real.jpg` (puppy, 256^2)	13,430 B	6,120 B (`real.brainimg`)	2.2×
`samples/test512.jpg` (512^2)	49,690 B	5,777 B (`test512.brainimg`)	8.6×
`samples/lenna.tiff` (512^2)	786,572 B	7,892 B (`lenna.brainimg`)	99.7×
(documented) `samples/real.jpg`	13.4 KB	2.7 KB (pre-seg v0.1)	5.0×

Two properties are worth noting. First, the blueprint size is **roughly constant** (a few KB) regardless of source resolution: a 256^2 and a 512^2 image both produce 5–8 KB files, because the stored maps are fixed at 128^2. Second, the compression ratio therefore *grows with source size*: the large uncompressed Lenna TIFF compresses ~100× while the already-compressed puppy JPEG compresses ~2×. This is the opposite of transform codecs, whose ratios are largely independent of whether the source is raw or pre-compressed, and is a direct consequence of storing meaning rather than pixels.

4.3 Reconstruction quality (qualitative)

Reconstruction is semantically faithful (same scene, layout, and lighting) but not pixel-identical—by design. The project ships side-by-side comparison assets, which we reference here as figures:

- `comparison.jpg` — original vs. brainimg reconstruction of `samples/real.jpg`. Per `README.md`, the captioner correctly described the scene ("a black puppy sitting on a wooden surface") and the decoder produced a visually faithful reconstruction at 256×256 in 59 s on the M1/8 GB machine.
- `lenna_comparison.jpg`, `test512_comparison.jpg` — 512^2 SD 1.5 reconstructions.
- `lenna_sdxl_comparison.jpg`, `lenna_sdxl.png` — an SDXL run on `lenna.brainimg` at 1024×1024 fp32, verified deterministic (identical md5 across runs, colour stats matched targets) per `TODO.md` Tier 3.
- `lenna_zimage_comparison.jpg`, `lenna_zimage.png` — a Z-Image-Turbo run on `lenna.brainimg` at 512×512 bf16 (8-step Turbo schedule; depth-only conditioning; canny + seg maps ignored per §3.3.2).
- `lenna_sdxl_512_comparison.jpg`, `lenna_sdxl_512.png` — SDXL at the same 512×512 resolution as the SD 1.5 default, allowing a like-for-like comparison (§4.7 below).

- `lenna_flux_depth_comparison.jpg`, `lenna_flux_depth.png` — FLUX.1-Depth-dev at 512×512 with FP8-quantized transformer + T5-XXL (`--quantize`). The first FLUX reconstruction committed to the repository.

4.4 Determinism

`README.md` reports that re-running the decoder with the same seed reproduces the same image exactly, verified as 0 pixel difference between runs. The SDXL Lenna run was additionally verified to produce an identical md5 across runs. Determinism is a property of the format: the seed is stored in the file, so a `.brainimg` file plus a fixed decoder yields a bit-identical image.

4.5 Device and precision ablation

Table 1 (§3.4) summarises the supported operating modes. The key engineering finding is that on Apple Silicon the SD 1.5 naive fp16 path is unusable (NaNs), forcing int8 weights + activations; this halves memory but degrades structural fidelity relative to CPU fp32. CPU fp32 is the recommended SD 1.5 mode on a high-RAM machine for best quality; CUDA fp16 is the recommended mode for speed. Z-Image-Turbo and FLUX are bf16 throughout (their native dtype), which sidesteps the MPS fp16 NaN bug entirely; FLUX additionally supports FP8 quantization of the transformer + T5 via `optimum.quanto` to halve resident memory at a small quality cost.

4.6 Known failure modes

Two failure modes remain documented in `TODO.md`:

- **Captioner misidentification.** The 7B captioner misidentifies Lenna's dark curled hair as *"a wide-brimmed straw hat adorned with purple feathers."* Because the conditioning maps (depth/Canny/seg) capture the true structure regardless, a wrong caption biases mood more than geometry—but it does bias generation. Mitigations noted for future work: a larger VLM or caption ensembling.
- **SDXL hue distribution drift at small sizes.** When SDXL is decoded at 512×512 on a source whose dominant palette is in a narrow band (e.g. Lenna's pink/magenta), the output lands in a different hue *distribution*: per-pixel hue histograms of the saturated pixels show SDXL@512 concentrating at 60°–90° (orange/yellow) instead of the source's 330°–30° (pink/magenta). This is a content/palette drift, not a stat drift: a single global HSV-H rotation aligns means but cannot reshape distributions, and a rotation large enough to chase a different distribution recolours neutrals and skin in a way that reads worse than the original drift. Workaround: prefer SDXL at 1024×1024, where the drift is much smaller (SDXL@1024 concentrates in the right band). FLUX.1-Depth-dev at 512×512 produces a less-drifted palette than SDXL@512 on the same blueprint (§4.7).

The brightness-clamp edge case that was previously listed here has been **fixed** with a gamma fallback (§3.3.1): when the uniform gain clamp $[0.5, 2.0]$ cannot reach an extreme target, a per-channel gamma curve closes the residual gap. Three new tests

in `tests/test_color.py` cover the gamma darkening, brightening, and approximate ratio preservation paths.

4.7 Per-backend fidelity and speed comparison (Lenna, 512×512)

We report three pixel-level metrics (MSE, PSNR, MAE) and wall time for each decoder backend on the Lenna blueprint (`samples/lenna.tiff`, 512×512, seed 916570520), all decoded at 512×512 on the AMD CPU target (188 GB RAM). Metrics are computed by `scripts/compare_lenna.py`. The SD 1.5 "old scales" row uses the historical defaults (1.5/1.2/0.9); the "tuned scales" rows use the new defaults (0.8/1.0/1.0) from the scale sweep (§4.8).

Backend	Steps	Time (s)	MSE ↓	PSNR (dB) ↑	MAE ↓
SD 1.5 (old scales)	30	~180	8763	8.70	77.5
SD 1.5 (tuned scales)	30	156	7560	9.35	70.8
SD 1.5 turbo (Hyper-SD, tuned)	8	**50**	7055	9.65	68.1
SDXL	30	220	5774	10.52	58.8
SDXL turbo (Hyper-SD)	8	**69**	6085	10.29	61.1
Z-Image (depth-only)	8	237	7651	9.29	70.3
Qwen-Image (depth-only)	50	1436	6810	9.80	68.4
FLUX.1-Depth-dev (FP8)	30	654	3202	13.08	43.6
FLUX.1-Depth-dev turbo (FP8)	8	**166**	**2314**	**14.49**	**37.1**

Five observations:

1. **Hyper-SD 8-step distilled schedules beat their 30-step counterparts.** On SD 1.5, the turbo path scores 9.65 dB vs 8.70 dB for the 30-step path with old scales (+0.95 dB) — at 3.5× less wall time. On FLUX, the turbo path scores 14.49 dB vs 13.08 dB for the 30-step path (+1.41 dB) — at 4× less wall time. The distilled schedule lands closer to the conditioning maps than the longer UniPC / FlowMatch schedule on this image. This is counter-intuitive (fewer steps usually means lower quality) and is a property of the distillation, not the base model.
2. **FLUX.1-Depth-dev turbo is the best result across all backends** (14.49 dB, 166 s) — the 8-step distilled FLUX beats both the 30-step FLUX and every other backend, while being 4× faster than the 30-step FLUX. The Hyper-SD FLUX LoRA was trained on base FLUX.1-dev, not the Control variants; the decoder strips the shape-incompatible `x_embedder` / `context_embedder` deltas (§3.3.4) but the attention/FFN deltas — the bulk of the distillation — load cleanly.
3. **ControlNet scale tuning adds +0.65 dB on the SD 1.5 30-step path** (8.70 → 9.35 dB) and +0.51 dB on the turbo path (9.14 → 9.65 dB, measured against the old-scale turbo baseline). The new defaults (depth 0.8, canny 1.0, seg 1.0) reflect that Depth-Anything-V2-Base's sharper depth map over-constrains at 1.5 (§4.8).
4. **Qwen-Image (Apache 2.0) is competitive with SDXL turbo** (9.80 dB vs 10.29 dB) despite being depth-only (no canny/seg), but slow at 50 steps (1436 s). It beats Z-Image (9.29 dB) on the same depth-only pattern. The Apache 2.0 license is a practical advantage over FLUX's non-commercial license for distribution.
5. **Z-Image is the weakest depth-only backend** (9.29 dB, 237 s) — slightly below SD 1.5 turbo (9.65 dB, 50 s) which uses three conditioning maps. Z-Image's photorealism advantage doesn't show up in pixel-level MSE on Lenna's narrow palette.

These numbers do not generalise beyond Lenna (a single source with a narrow pink palette); §5.3 notes that a real evaluation requires more subjects and perceptual metrics (LPIPS, CLIP-score, FID). The combined side-by-side grid of all backends is in `lenna_grid.jpg`.

4.8 ControlNet scale sweep

The historical SD 1.5 conditioning defaults (depth 1.5, canny 1.2, seg 0.9, cfg 7.5) were set for the older Depth-Anything-Small + no-seg pipeline. With Depth-Anything-V2-Base (sharper depth) and the ADE20K seg ControlNet now in the stack, we ran a grid sweep on `samples/lenna.tiff` and `samples/test512.jpg` at 512×512 with `sd15-turbo` (8 steps, seed from the blueprint), using `scripts/sweep_lenna.py` — 3 passes, ~35 configurations, loading the pipeline once and varying only the scale/cfg pair.

Findings:

- **Lower depth helps.** Depth 1.0 beats 1.5 on both samples; 0.8 beats 1.0; 0.6 beats 0.8 on test512 but hurts Lenna. The V2 depth map is sharper than the Small model's, so high scales over-constrain and fight the caption. The sweet spot is 0.8 (good on both).

- **Higher seg helps.** Seg 1.2 beats 0.9 on Lenna; 0.9 beats 1.2 on test512. The ADE20K seg ControlNet adds material cues that were missing in the old no-seg pipeline; parity with canny (1.0) is the compromise.
- **Canny 1.0 beats 1.2.** A small but consistent improvement on both samples.

New SD 1.5 defaults: depth 0.8, canny 1.0, seg 1.0, cfg 7.5 (was 1.5/1.2/0.9/7.5). Measured lift on Lenna: SD 1.5 turbo 9.14 → 9.65 dB (+0.51 dB from scales alone, on top of the +0.44 dB the distilled schedule already contributed vs the 30-step path). SDXL defaults (1.0/0.8/0.6) were left unchanged — they were already in the good region.

4.9 MAP_SIZE regression

A TODO item proposed raising `MAP_SIZE` from 128 to 256 for sharper conditioning maps. We tested this on Lenna at 512×512 and it **regressed on every backend**: SD 1.5 30-step −0.65 dB (8763 → 10185 MSE), SD 1.5 turbo −0.85 dB (7934 → 9640), SDXL turbo −0.57 dB (6085 → 6928). File size also grew 2.5× (7.9 KB → 19.7 KB). The ControlNets appear over-constrained by the sharper maps at 512×512 output — 128 maps upscaled 4× to 512 give the right amount of structural grip, while 256 maps upscaled 2× over-specify edges/depth. `MAP_SIZE` stays at 128; the finding is documented in `TODO.md` and `format.py`. Re-evaluation is warranted if the default output size moves to 1024.

5. Discussion

5.1 Properties of the paradigm

Resolution independence. A `.brainimg` file has no native resolution: it is a set of instructions for a generative model, so the same few-KB file can be decoded at 256, 512, 1024, or higher, the decoder simply generating more pixels. This is qualitatively different from transform codecs, where up-sampling a small file produces a blurry large image.

Semantic, not pixel, loss. In JPEG, loss is block artefacts and colour bleeding. In brainimg, "loss" means the model may render a slightly different wood grain or fabric pattern than the original, while the fabric still looks photorealistic. The lost data is micro-detail, replaced by synthesised micro-detail. This is desirable for human-visual consumption and unacceptable for forensic/medical/legal use where exact pixels matter.

Editability. Because the file is a self-describing JSON blueprint, a user can edit the caption, mood prefix, or conditioning scales and re-decode to relight or re-pose the scene—an early hint of the "edit the recipe, not the pixels" property of the Semantic-Relational Field paradigm.

Determinism. Stored seed + fixed decoder ⇒ bit-identical re-decode. This addresses the usual "hallucination is non-deterministic" objection to generative codecs: within a fixed decoder version, a file *is* a stable image.

5.2 Limitations

- **Decoder dependency.** Unlike JPEG's kilobyte decoder, brainimg's decoder is a multi-gigabyte neural network (anywhere from ~4 GB for SD 1.5 to ~22 GB for FLUX.1 bf16, reducible to ~12 GB with FP8 quantization). A device without the standard decoder cannot view the image—the codec is more like a video codec in this respect. On the other hand, the decoder is *shared* across all files, so the per-file cost remains a few KB.
- **Compute cost.** Decompression runs a diffusion model: 50 s for SD 1.5 turbo at 512^2 on the AMD CPU target, 166 s for FLUX turbo (FP8), ~24 min for Qwen-Image at 50 steps. The turbo paths make interactive use plausible on CPU; the 30-step paths remain in the minutes-per-image range. This still precludes gallery scrolling or video use cases.
- **Quality depends on device.** CPU fp32 gives the best SD 1.5 reconstruction; on 8 GB Apple Silicon, int8 quantization (required by the MPS fp16 NaN bug) degrades structural fidelity. Z-Image-Turbo, Qwen-Image, and FLUX are bf16-native and are not supported on 8 GB Apple Silicon; FLUX CPU requires ~22 GB host RAM (or ~12 GB with `--quantize`).
- **Lossy by design.** Reconstruction is semantically faithful, not pixel-identical. The format is explicitly unsuited to medical, legal, or forensic images.
- **Captioner accuracy.** Wrong captions bias generation (§4.6). Structure is the primary fidelity driver, which limits but does not eliminate the impact of caption errors.
- **Per-backend palette fidelity.** SDXL and Z-Image, run at sizes smaller than their native training resolution (1024), can drift into a different hue *distribution* than the source (§4.6). This is a content/palette drift the existing brightness/saturation post-processor cannot correct. Prefer each backend's native resolution (512 for SD 1.5, 1024 for SDXL / Z-Image / FLUX) when colour fidelity matters.
- **License footprint.** The default backend (SD 1.5 + ControlNets) is CreativeML Open RAIL-M. SDXL is the same. Z-Image-Turbo is non-commercial. FLUX.1-Depth-dev / -Canny-dev are FLUX.1-dev non-commercial. Qwen-Image is **Apache 2.0** — the only fully-open high-quality option. If distribution matters, the SD 1.5/SDXL and Qwen-Image paths are the permissively licensed choices.

5.3 Planned evaluation (not run here)

A complete evaluation would include: PSNR/SSIM/LPIPS against originals for the sample images across all backends and device modes; CLIP-score as a semantic-fidelity proxy; FID against a small natural-image set; a bitrate-matched comparison to JPEG/WebP/AVIF at similar file sizes; and an ablation removing each ControlNet in turn to quantify each conditioner's contribution. The current measurements are on a single image (Lenna) with a narrow pink palette; a real evaluation requires more subjects. The repository is structured to support these via `encoder.py`/`decoder.py` and the `samples/` directory.

6. Conclusion

brainimg is a small, reproducible prototype of a different way to compress images: store the *meaning and structure* of a scene and let a diffusion model repaint it. We have described the format schema, the four-stage encoder, the ten decoder backends (SD 1.5, SDXL, their Hyper-SD turbo variants, Z-Image-Turbo, Qwen-Image, FLUX.1-Depth-dev / -Canny-dev and their Hyper-SD turbo variants) with their quality post-processing and gamma brightness fallback, and the device/precision tradeoffs across CPU fp32, MPS int8, and CUDA fp16. Using measurements reproducible from the committed repository on an AMD CPU target with 188 GB RAM, we observe few-kilobyte blueprints (2.2×–99.7× compression), deterministic reconstruction given a seed, and two counter-intuitive empirical findings: (1) Hyper-SD's 8-step distilled schedules *beat* their 30-step counterparts on both SD 1.5 (+0.95 dB) and FLUX (+1.41 dB) while running 3.5–4× faster on CPU, and (2) lower ControlNet depth scales (0.8 vs 1.5) improve fidelity with the Depth-Anything-V2-Base stack. FLUX.1-Depth-dev turbo (FP8, 8 steps) is the best result across all backends at 14.49 dB PSNR and 166 s. We frame this as a systems study of a paradigm, not a JPEG replacement, and we are explicit about its limitations—decoder dependency, compute cost, per-backend palette fidelity, and lossy-by-design semantics.

Planned work tracked in `TODO.md` includes per-region hue transfer (a fix for the SDXL hue-distribution drift documented in §4.6) and a captioner upgrade (§4.6). The MAP_SIZE 128→256 bump, ControlNet scale tuning, brightness-clamp gamma fix, and all turbo distillation backends are done.

7. Reproducibility

The project is self-contained and engineered to run on commodity hardware. Setup (Python 3.12 via uv):

```
uv venv -p 3.12
source .venv/bin/activate
uv pip install -r requirements.txt
# On non-Apple platforms, uninstall the non-functional MLX stub wheels:
pip uninstall -y mlx mlx-vlm mlx-lm
```

Encode and decode:

```
# photo -> tiny .brainimg blueprint
python encoder.py samples/real.jpg -o out.brainimg --seed 42

# blueprint -> regenerated image (CPU fp32, best fidelity)
python decoder.py out.brainimg -o recon.png --device cpu

# CUDA fp16 (fast), or CPU int8 (low RAM)
python decoder.py out.brainimg -o recon.png --device cuda
python decoder.py out.brainimg -o recon.png --device cpu --quantize
```

The format and colour post-processing modules are deliberately ML-free and can be

tested without downloading models:

```
pytest            # tests/test_format.py + tests/test_color.py + tests/test_flux_config
```

Known gotchas (from `AGENTS.md`): the MPS fp16 NaN bug is *not* a bug to "fix" by switching to fp16—int8 quantization is the intended workaround (Apple Silicon only; not relevant on the AMD CPU target). The Hyper-SD FLUX LoRA was trained on base FLUX.1-dev, not the Control variants; the decoder strips the shape-incompatible `x_embedder` / `context_embedder` deltas before loading (§3.3.4). Re-decoding with the same seed reproduces the image exactly.

References

The references below are to canonical works and projects used by brainimg. They are given in a lightweight author–year–title form suitable for a Markdown draft; a submission version would expand them into a `.bib`.

- Rombach, R., Blattmann, A., Lorenz, D., Esser, P., Ommer, B. (2022). *High-Resolution Image Synthesis with Latent Diffusion Models* (Stable Diffusion). CVPR 2023.
- Zhang, L., Rao, A., Agrawala, M. (2023). *Adding Conditional Control to Text-to-Image Diffusion Models* (ControlNet). ICCV 2023.
- Gu, Z., Wang, W., Huang, Z., Chen, J., Dong, Z., Zhang, W., et al. (2024). *Depth Anything V2* (Depth-Anything-V2-Base).
- Jain, J., Li, J., Chiu, M.-T., et al. (2023). *OneFormer: One Transformer to Rule Universal Image Segmentation* (OneFormer ADE20K).
- Wang, P. et al. (2024). *Qwen2-VL / Qwen2.5-VL Technical Report* (vision-language captioning).
- Apple MLX framework and `mlx-vlm` (Apple Silicon 4-bit captioning).
- HuggingFace `diffusers` library; `transformers` library.
- HuggingFace `optimum-quanto` (int8 quantization for MPS/CPU).
- `stabilityai/sd-vae-ft-mse` (fine-tuned SD 1.5 VAE).
- `stabilityai/stable-diffusion-xl-base-1.0`; `diffusers/controlnet-{depth,canny}-sdxl-1.0`; `abovzv/sdxl_segmentation_controlnet_ade20k` (SDXL stack).
- `lllyasviel/control_v11f1p_sd15_depth`, `lllyasviel/control_v11p_sd15_canny`, `lllyasviel/control_v11p_sd15_seg` (SD 1.5 ControlNets).
- `Tongyi-MAI/Z-Image-Turbo` (6 B-parameter DiT distilled for 8-step inference) with `alibaba-pai/Z-Image-Turbo-Fun-Controlnet-Union-2.1` (the full `2.1-8steps` variant; the `*lite*` 2601/2602 files are rejected by diffusers 0.38 due to a widened `control_all_x_embedder` input dim).
- `black-forest-labs/FLUX.1-Depth-dev` and `black-forest-labs/FLUX.1-Canny-dev` (FLUX.1 guidance-distilled Control variants; channel-concat conditioning). `black-forest-labs/FLUX.1-dev` (the underlying 12 B MMDiT + T5-XXL base) — gated on Hugging Face; both FLUX Control variants carry the FLUX.1-dev non-commercial license.
- Ren, Y., Xia, X., Lu, Y., et al. (2024). *Hyper-SD: Trajectory Segmented Consistency Model for Efficient Image Synthesis* (arXiv 2404.13686). ByteDance/Hyper-SD LoRAs for SD 1.5, SDXL, and FLUX.1-dev (8-step distillation).

- Wu, C., Li, J., Zhou, J., et al. (2025). *Qwen-Image Technical Report* (arXiv 2508.02324). `Qwen/Qwen-Image` (Apache 2.0 DiT) with `InstantX/Qwen-Image-ControlNet-Union` (Union ControlNet, Apache 2.0).
- `peft` (Parameter-Efficient Fine-Tuning library, HuggingFace) for LoRA loading on the turbo paths.
- Wallace, G. K. (1992). *The JPEG Still Picture Compression Standard.* Communications of the ACM.
- WebP / VP8 intra-frame specification (Google).
- AVIF / AV1 intra-frame specification (Alliance for Open Media).
- JPEG 2000 (wavelet/fractal) standard.
- Olshausen, B. A., Field, D. J. (1996). *Emergence of simple-cell receptive field properties by learning a sparse code for natural images* (sparse coding / brain motivation).
- Rao, R., Ballard, D. (1999). *Predictive coding of natural images* (predictive coding / brain motivation).
- Hubel, D., Wiesel, T. (1959/1962). *Receptive fields of single neurons in the cat's striate cortex* (V1 feature extraction motivation).
- Classic retinal compression / lateral inhibition accounts (visual neuroscience, motivational background only).